GRK Test 1 · study guide

lectures 1–6 · Fri 13 Nov 2026, 08:30 · 15 points

This guide teaches each topic in the same order: **big idea → formulas → how to remember → every task type with its recipe, a mini example, the traps and the by-hand shortcut**. Practise with the separate *Tasks and answer key* PDF; check yourself in the phone trainer (anastasia-d-kravchenko.github.io/grk-test1-trainer).

The test

- Open book: your own handwritten or printed notes and books, no sharing.
- **No electronic devices** — no calculator, no phone. Practise every number by hand.
- Typical paper: 3–4 tasks, each a calculation + drawing + "describe the steps".

Answer protocol (every task)

1. **Formula** first — it already earns partial credit.
2. **Table of steps** (current, p, decision, next…).
3. **Drawing** on the grid, coordinates read twice.
4. **Boxed answer** with units, plus your **convention** where a choice exists.

How to study (what works)

- Each block: 5 min read the recipe (Tasks tab) → do the official example with the answer covered (pack Part B) → 2–3 random problems in Solve with the solution hidden → 10 min cards for that topic.
- Never look at a solution before you try. Mixing topics later (mock, cards on "All") is what makes it stick.
- Every task on paper: formula → table of steps → drawing → boxed answer, and write your convention.
- No calculator in the test (no electronic devices): practise every calculation by hand today, using the by-hand hints in Tasks and the tables in pack Part C.
- **Retrieval** beats re-reading: cover the answer, produce it, then check. **Spacing**: come back to a topic after a break. **Interleaving**: mix topics in the mock. **Worked → faded**: follow a solved example, then do one with new numbers alone.

Lecturer conventions

topic	convention
Data rate 4K	slide: 4096×2160 ≈ 10^7 ticks → $10^7 \cdot 3 \cdot 60 = 1.8 \cdot 10^9$ B/s; write the exact $1.59 \cdot 10^9$ too
Megabyte	MB = 10^6 B (slide); also give MiB = 2^{20} B
Eye limit	tan 1′ = 0.00029 (slide) → 2069; exact 0.0002909 → 2063; n = W/(D·tan 1′)
8 → n bits	floor(v / 2^(8−n)) = drop the low bits; state it
Grey	weighted 0.299R + 0.587G + 0.114B, and the average (R+G+B)/3
DDA	step x by 1, y += m, round; |m| ≤ 1
Bresenham	start at the smaller x; $p_0 = 2\Delta y - \Delta x$; p ≥ 0 (incl. 0) → up
Midpoint circle	start (0,r), $p_0 = 5/4 - r$; update with the NEW x, y; p ≥ 0 → y−1; stop at x ≥ y; mirror 8 ways
Seed fill	recursive, left, right, up, down; name the seed
Scan-line	crossings, sort, fill pairs; peak/valley vertex counts twice; horizontal edges = contour
Barycentric	α+β+γ = 1; inside ⇔ all ≥ 0; value = αf_A + βf_B + γf_C
Transforms	column vectors, first transform = rightmost: M = last·…·first
Frame change	rows of the matrix = new axes; frame rotated +θ = point rotated −θ
Cohen–Sutherland	code TBRL (1001 = top-left); both 0000 accept; AND ≠ 0 reject; cut edges in order left, top, right, bottom
Sutherland–Hodgman	one window edge at a time (left, top, right, bottom); out→in: P, W; in→in: W; in→out: P; out→out: nothing
Bézier	degree = points − 1; through P_0 and P_n only; C1 join: $Q_1 = 2P_n - P_{n-1}$

Wrong answers circulating (2025 guide)

guide says	correct
pixel time "6 µs"	6.03 ns
3 bits/channel "8 colours"	512
circle p ≥ 0 → "y+1"	y−1
frame change "R·P = (−2,2)"	(2,−2)
Bézier with P_1 = (10,1)	control point P_1 = (4,8)
grey "R + G + B/3"	(R+G+B)/3

L1 · Displays, pipeline, GPU

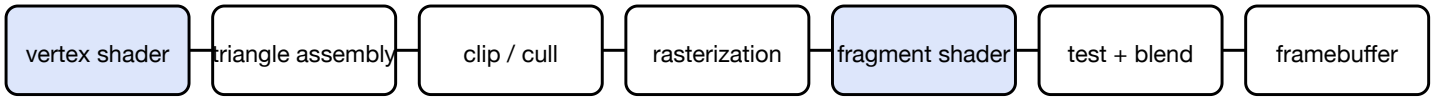
Big idea. A raster screen shows a frame buffer pixel by pixel, row by row, many times per second. The GPU builds that buffer through a pipeline: vertices are transformed (vertex shader), assembled into triangles, clipped, rasterized into fragments, coloured (fragment shader), depth-tested and written to the frame buffer.

Formulas

- `data rate = W·H·(bytes/px)·Hz`
- `time per pixel = 1 / (W·H·Hz)`
- `vector memory = Σ numbers · 4 B (circle 3, triangle 6)`

Remember it

- Pipeline order: "Very Pretty Rabbits Fetch Pixels" = Vertex shader → Primitive assembly + clip → Rasterize → Fragment shader → Per-fragment ops (depth/blend) → frame buffer.
- Vertex decides WHERE, fragment decides WHAT COLOUR.
- Double buffering = ping-pong: draw into the back buffer, show the front, swap.



blue = programmable · vertices → primitives → fragments → pixels

Data rate of a display P1 Q1

What is the data rate (bytes/s) needed to display a 4K image at 60 Hz?

1. pixels = W × H
2. bytes per pixel = bits/8 (24-bit → 3 B)
3. rate = pixels × bytes × Hz
4. give B/s and GB/s; mention the lecture's ≈10^7 ticks rounding

Example: Slide 26: 4K = 4096×2160 ≈ 10^7 ticks (with blanking) → $10^7 \times 3 \times 60 = 1.8 \cdot 10^9$ B/s ≈ 1.7 GiB/s. Pixels only: 4096·2160·3·60 = $1.59 \cdot 10^9$ B/s.

Traps: bits vs bytes · forgetting × Hz

By hand: Keep powers of ten apart: 4096·2160 ≈ $8.85 \cdot 10^6$ (slide rounds to 10^7). Bytes × Hz first: 3·60 = 180. Then $10^7 \cdot 180 = 1.8 \cdot 10^9$ B/s. For 1920·1080 use the resolution table (2,073,600).

Time to display one pixel P1 Q2

1920×1080 at 80 Hz: time needed to display a single pixel?

1. pixels per second = W·H·Hz
2. t = 1 / that
3. convert to ns (× 10^9)

Example: 1/(1920·1080·80) = 6.03 ns

Traps: answering per frame (12.5 ms) instead of per pixel

By hand: W·H from the resolution table, × Hz → write it as $a \cdot 10^n$ (1920·1080·80 = $1.65888 \cdot 10^8$). Then 1/a from the reciprocal table: 1/1.659 ≈ 0.603 → $0.603 \cdot 10^{-8}$ s = 6.03 ns.

Memory of a vector image P1 Q3

Memory for 100 circles and 100 triangles, all numbers 4-byte floats?

1. list the numbers each shape needs
2. circle: x, y, r = 3; triangle: 3 × (x, y) = 6
3. multiply by 4 B and by the count, add

Example: 100·12 + 100·24 = 3600 B

Traps: storing pixels instead of numbers · say your rectangle convention (4 or 8 numbers)

By hand: All integers: numbers per shape × 4 B × count. Circle 12 B, triangle 24 B, rectangle 32 B, cube 96 B: just multiply by 100.

Stages of the graphics pipeline P1 Q4

Describe the stages of the graphics pipeline.

1. Application (CPU): scene, culling, draw calls
2. Vertex shader: transform vertices (model→view→projection), per-vertex data
3. Primitive assembly → clipping → (back-face) culling
4. Rasterization: triangles → fragments (interpolated values)
5. Fragment shader: colour of each fragment (texture, lighting)
6. Per-fragment ops: depth test, blending → frame buffer

Example: Draw it as a row of boxes with arrows; label which are programmable (vertex, fragment).

Traps: swapping vertex and fragment shader roles

By hand: Theory only.

Fragment shader input and output P1 Q5

What is the input and output data of the fragment shader?

1. IN: values interpolated by the rasterizer for this pixel (colour, texture coords UV, normal, position/depth) + uniforms (textures, lights, matrices)
2. OUT: the fragment colour (RGBA), optionally depth

Example: One sentence each for input and output, plus "runs once per fragment".

Traps: saying it outputs vertex positions (that is the vertex shader)

By hand: Theory only.

Watch: How do Video Game Graphics Work? (Branch Education, 21:00) · Triangles to Pixels (Computerphile, 8:16) · Vertex and Fragment Shaders (Udacity, 1:35)

L1 · Vectors and matrices

Big idea. A vector AB = B − A. Add vectors tip-to-tail. The cross product a × b is perpendicular to both (right-hand rule) and its length is the parallelogram area. A matrix times a vector is "each row · the vector".

Formulas

- `a × b = [a_y b_z − a_z b_y, a_z b_x − a_x b_z, a_x b_y − a_y b_x]`
- `x × y = z, y × z = x, z × x = y`
- `b × a = −(a × b)`

Remember it

- Cross product "xyzzy": x-component uses y,z; then shift letters cyclically.
- Right-handed system: x right, y up, z toward YOU.

Vectors: components, sum, cross product, matrix × vector P1

Q28–31

A(1,1,1), B(3,1,1), C(1,2,1): draw AB, AC and write components. X = [−2,0,0], Y = [0,1,0] from P(−1,1,0): sum, cross product. Multiply 3I·[2,−1,0].

1. AB = B − A
2. sum: tip-to-tail, components add
3. cross: determinant formula, check with right-hand rule
4. matrix × vector: row · vector
5. draw in a right-handed system (z toward you)

Example: AB = [2,0,0], AC = [0,1,0]; X+Y = [−2,1,0] ends at (−3,2,0); X×Y = [0,0,−2]; 3I·[2,−1,0] = [6,−3,0] (scaling by 3).

Traps: X × Y vs Y × X sign · drawing from the origin instead of from P

By hand: Integer arithmetic only. Cross product with the "xyzzy" pattern; check the direction with the right-hand rule. Matrix × vector = row · vector, one row at a time.

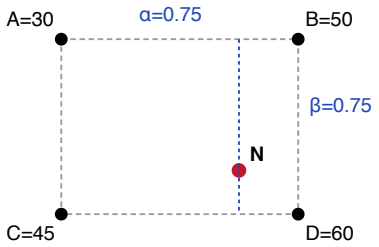
Watch: Cross products (3Blue1Brown, 8:53) · Cross and Dot Product: visual (Eugene Khutoryansky, 4:07)

L2 · Image, resolution, resampling

Big idea. A raster image is a grid of samples (sampling = where; quantization = which values). Pixel resolution is W×H; linear resolution is pixels per inch. Memory = W·H·bits/8. Changing resolution means computing new samples: average blocks to shrink, interpolate (bilinear) to enlarge or shift.

Formulas

- `memory = W·H·bits / 8 B (MB = 10⁶, MiB = 2²⁰)`
- `ppi = px / inches; 16:9 width = d·16/√337`
- `eye: x = D·tan(1′), n = W/x`
- `bilinear: P1 = αB+(1−α)A, P2 = αD+(1−α)C, N = βP2+(1−β)P1`



Remember it

- Bilinear = "lerp, lerp, then lerp the lerps".
- The eye sees 1 arc-minute: about 0.3 mm at 1 m, 0.15 mm at 50 cm.
- Undersampling = the wagon wheel spinning backwards: detail smaller than the new pixel disappears or turns into false patterns.

Memory of a bitmap P1 Q6

Memory to store an image 1024 × 768 × 24, in MB.

1. bits = W·H·depth
2. ÷ 8 → bytes
3. ÷ 10^6 → MB (or ÷ 2^{20} → MiB; write which)

Example: 1024·768·24 = 18,874,368 bits = 2,359,296 B = 2.36 MB (2.25 MiB)

Traps: bits vs bytes · MB vs MiB unstated

By hand: Divide by 8 early: 24 bit = 3 B, 16 bit = 2 B. W·H from the resolution table, ×3. MB = move the comma 6 places (2,359,296 B = 2.36 MB).

Linear resolution (ppi) of a display P1 Q7

42″ diagonal, 1920×1080, 16:9. Linear resolution?

1. width = diag · 16/√(16²+9²) = diag·16/√337
2. ppi = horizontal pixels / width in inches
3. (check with height)

Example: width = 36.61″ → 1920/36.61 = 52.45 ppi

Traps: dividing pixels by the diagonal

By hand: width = diagonal × 0.8716 for 16:9 (×0.8 for 4:3; table). 42·0.8716: 42·0.87 = 36.54, +42·0.0016 = 0.07 → 36.61. ppi = 1920/36.61: 36.61·50 = 1830.5, rest 89.5 ≈ 2.45·36.61 → 52.45.

Maximum useful resolution (human eye) P1 Q8

Display 30 cm wide viewed from 50 cm. Max horizontal resolution the eye can use?

1. eye resolves 1′ = 1/60°
2. x = D·tan(1/60°)
3. n = W / x

Example: x = 50·tan(1′) = 0.01454 cm → 30/0.01454 ≈ 2063 px (slide 20 rounds tan 1′ ≈ 0.00029 → x = 0.0145 → ≈ 2069)

Traps: using degrees instead of arc-minutes · mixing cm and mm

By hand: n = W/(D·tan1′) = (W/D)·3438 (1/tan 1′ = 3437.7). 30/50 = 0.6 → 0.6·3438 = 2063. With the slide's 0.00029: (W/D)·3448 = 2069.

Pseudo-code: decrease resolution 2:1 P1 Q9

Write a function that halves (or doubles) the resolution of Color[100][100].

1. new array 50×50
2. each new pixel = average of the 2×2 block [2y..2y+1][2x..2x+1]
3. average each channel R, G, B separately

Example: Color[][] small = new Color[50][50];
for y in 0..49: for x in 0..49:
small[y][x] = avg(img[2y][2x], img[2y][2x+1], img[2y+1][2x], img[2y+1][2x+1])

Traps: just taking every second pixel (that is nearest-neighbour; allowed but loses info, say so)

By hand: No numbers: write the 2×2 average loop. Mention the per-channel division by 4.

Resampling: intensity of a new pixel (bilinear) P1 Q16

New pixel N lies between old pixels A=30, B=50, E=45, F=60. Intensity of N?

1. read α (horizontal) and β (vertical) from the grid: distance of N's centre from A's centre in old-pixel units
2. P1 = αB + (1−α)A
3. P2 = αF + (1−α)E
4. N = βP2 + (1−β)P1

Example: α = β = 0.75: P1 = 45, P2 = 56.25, N = 53.44

Traps: measuring from pixel corners instead of centres · swapping α with 1−α

By hand: Use fractions, not decimals: α = ¾ → P1 = (A + 3B)/4; β = ¾ → N = (P1 + 3P2)/4. (30 + 150)/4 = 45; (45 + 180)/4 = 56.25; (45 + 168.75)/4 = 53.44.

Sketch the image resampled to lower ppi P1 Q27

6×6 image sampled at 10 ppi; sketch it at 5 ppi. Good representation?

1. 5 ppi = each new pixel covers 2×2 old pixels
2. new value = average of the block (draw greys)
3. judge: detail finer than 2 old pixels is lost (undersampling/aliasing)

Example: A 1-px pattern turns every block into the same grey → detail lost → bad representation.

Traps: answering "yes, it is fine"

By hand: Count dark pixels in each 2×2 block: 0, 1, 2, 3, 4 dark → 0 %, 25 %, 50 %, 75 %, 100 % grey.

Watch: Resizing Images (Computerphile, 9:21) · What is "Retina"? PPI (Techquickie, 3:23) · Digital Images (Computerphile, 8:15)

L3 · Colour models

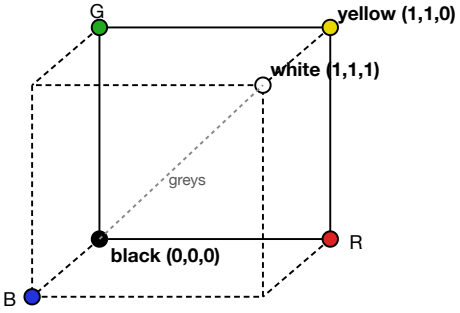
Big idea. RGB is additive light (unit cube, black at the origin, white opposite). CMY is subtractive ink (= 1 − RGB). HSV describes colour as hue angle, saturation and value in a cone. Grey = weighted sum (green counts most). CIE xy chromaticity diagram: pure spectral colours on the horseshoe edge; any mix of two colours lies on the segment between them.

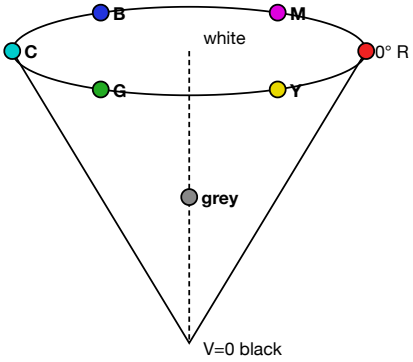
Formulas

- `colours = (2ᵇ)³`
- `8→n bits: floor(v / 2^(8−n))`
- `CMY = 255 − RGB`
- `grey = 0.299R + 0.587G + 0.114B`
- `mix 50/50 = average of channels`
- `V = max, S = (max−min)/max, H by which channel is max`

Remember it

- "Cyan eats Red, Magenta eats Green, Yellow eats Blue."
- Hue clock: R 0°, Y 60°, G 120°, C 180°, B 240°, M 300° ("every 60, RYGCBM").
- Greys live on the axis of the HSV cone; pure colours on the rim of the top disc; black is the tip.
- Two lights mix on the straight line between them; three primaries make a triangle (the gamut).





How many colours for b bits per channel P1 Q10, EXAM 1/23

How many distinct colours with 3 bits per RGB channel?

1. levels = 2^b per channel
2. colours = $(2^b)^3$

Example: $(2^3)^3 = 512$

Traps: answering $2^3 = 8$ (that is per channel)

By hand: Powers-of-2 table: $(2^b)^3 = 2^{(3b)}$. 3 bits → $2^9 = 512$; 4 → $2^{12} = 4096$; 8 → $2^{24} = 16{,}777{,}216$.

Reduce colour/grey depth (8 → n bits) P1 Q11, EXAM 1/23 #1

Grey levels at 8 bits are given; compute them at 4 bits. / RGB colours at 8 bits → 3 bits per channel.

1. divide by 2^(8−n), floor (= drop the low bits)
2. do every channel
3. comment: which values merge (objects become indistinguishable)

Example: 143, 240, 128 → 8, 15, 8 (circle merges with background). [242,254,4] → [7,7,0].

Traps: rounding instead of floor (state your rule) · forgetting to say which ones merge

By hand: No division needed: find the value's bin in the "8 → n bits" table (÷32 for 3 bits: 0–31 → 0, 32–63 → 1 … 224–255 → 7; ÷16 for 4 bits).

RGB → CMY P1 Q12

Convert [255,135,0], [0,0,0], [255,255,255] to CMY.

1. C = 255 − R, M = 255 − G, Y = 255 − B

Example: [0,120,255], [255,255,255], [0,0,0]

Traps: forgetting black ↔ white swap

By hand: 255 − v: count up from v to 255 (135 → 255 is 120).

RGB → greyscale P1 Q13

Convert [128,0,128], [255,255,0] to grey.

1. Y = 0.299R + 0.587G + 0.114B
2. ALSO write the average (R+G+B)/3: the slide shows both and the question may expect either
3. round, and say which one is your final answer

Example: weighted 53, 226 · average 85, 170

Traps: R+G+B/3 without brackets

By hand: Weight table (0.299v, 0.587v, 0.114v) for round values, or approximate 0.3R + 0.59G + 0.11B (off by < 1). For 128: 0.299·128 = 38.27, 0.114·128 = 14.59 → 52.86. The average (R+G+B)/3 is easy: write both.

Chromaticity diagram: what can be mixed P1 Q14

Two colours 1 and 2 are marked on the xy diagram. Which colours can be obtained by mixing them?

1. all colours on the straight segment 1–2
2. the share decides the position (more of 1 → closer to 1)
3. mention: three primaries → their triangle = gamut; spectral colours on the curved edge

Example: Draw the segment between 1 and 2 and shade it.

Traps: shading a region instead of a segment

By hand: No numbers: draw the straight segment between 1 and 2.

Mix two RGB colours P1 Q15

Mix A = [255,255,0] and B = [0,255,255] in equal proportions.

1. each channel = (a + b)/2
2. round

Example: [127.5, 255, 127.5] ≈ [128,255,128] (light green)

Traps: adding and clamping to [255,255,255]

By hand: (a + b)/2 per channel; 255/2 = 127.5 → 128. For 25/75: (a + 3b)/4.

Describe the RGB model P1 Q17

Components, coordinate system (draw axes), mark white/black/yellow with numbers, applications.

1. components: red, green, blue light intensities (additive)
2. unit cube, axes R, G, B; black at origin, white (1,1,1) opposite
3. yellow (1,1,0) = R+G; cyan (0,1,1); magenta (1,0,1)
4. grey diagonal from black to white
5. applications: monitors, cameras, scanners, projectors (light-emitting devices)

Example: In 8-bit: black [0,0,0], white [255,255,255], yellow [255,255,0].

Traps: saying printers use RGB (they use CMY(K))

By hand: No numbers: draw the cube with the 8 vertices labelled.

Mark colours in the HSV cone EXAM 1/23 #1c

Mark the resulting colours in the HSV cone.

1. V = max, S = (max−min)/max, H from which channel is max
2. draw the cone: tip = black, axis = greys, top disc = V = 1, angle = hue
3. place each point; fully saturated bright colours are on the top rim

Example: [7,7,0] yellow H=60°, S=1, V=1 → rim of the top disc; [7,0,7] magenta H=300° → rim.

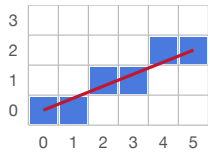
Traps: putting greys on the rim

By hand: V = max/255. S = (max − min)/max (1 if min = 0). H: pure mixes from the hue clock (R 0, Y 60, G 120, C 180, B 240, M 300); otherwise H = sector start ± 60·(mid − min)/(max − min). [255,128,0]: 60·128/255 ≈ 60·0.5 = 30°.

Watch: The HSV colour model (Thales Sehn Körting, 2:25) · RGB vs CMYK (PiXimperfect, 4:48) · CIE chromaticity diagram (Khan Academy / Pixar, 1:33) · Colour spaces from the ground up (Video Tech Explained, 16:24)

L4 · Lines and antialiasing

Big idea. For a gentle line (|m| ≤ 1) light exactly one pixel per column. DDA adds the slope m each step and rounds (floats). Bresenham keeps an integer decision value p that says "stay in this row" (p < 0) or "go up" (p ≥ 0). Antialiasing lights the two nearest pixels per column with intensity by distance.



Formulas

- `DDA: y += m, plot round(y)`
- `Bresenham: a = 2Δy, b = 2Δy − 2Δx, p₀ = 2Δy − Δx`
- `p < 0 → (x+1, y), p += a`
- `p ≥ 0 → (x+1, y+1), p += b`

Remember it

- DDA = "Do Decimal Addition" (floats).
- Bresenham: "Negative: stay, pay a. Non-negative: step up, pay b." Start from the smaller x.
- Steeper than 45° → swap the roles of x and y, or you get gaps.
- Aliasing cannot be removed, only reduced.

DDA: steps and pixels P1 Q18

Describe the steps of DDA and mark pixels from (0,0) to (5,3).

1. m = Δy/Δx (check |m| ≤ 1)
2. start y = y_0; each column x+1: y += m
3. plot (x, round(y))
4. table: x | y | round(y)

Example: (0,0) (1,1) (2,1) (3,2) (4,2) (5,3)

Traps: not rounding half up the same way each time

By hand: Keep m as a fraction (3/5): y = 0, 3/5, 6/5, 9/5, 12/5, 3 → round by comparing the remainder with half the denominator (3/5 ≥ ½ → up).

Why slope ≤ 45° in basic DDA P1 Q19

Why the assumption |m| ≤ 1? Show an example.

1. DDA steps x by 1 → one pixel per column
2. if |m| > 1, y jumps by more than 1 → gaps
3. example (0,0)→(2,5): (0,0), (1,3), (2,5) only
4. fix: for steep lines step y and add 1/m to x

Example: Draw the 3 isolated pixels and the gaps.

By hand: No arithmetic: show 3 isolated pixels for (0,0)→(2,5).

Bresenham: steps and pixels P1 Q20, Gracki tests

Describe the steps of Bresenham and mark pixels from (0,0) to (5,2).

1. start at the endpoint with the smaller x
2. Δx, Δy; a = 2Δy, b = 2Δy − 2Δx, p_0 = 2Δy − Δx
3. p < 0 → same row, p += a; p ≥ 0 → up, p += b
4. table: current | p | decision | next

Example: a=4, b=−6, p: −1, 3, −3, 1, −5 → (0,0) (1,0) (2,1) (3,1) (4,2) (5,2)

Traps: p ≤ 0 instead of p < 0 · starting from the right end

By hand: Integers only, made for no calculator: a, b, p_0, then add a or b each step.

Antialiased version of a line P1 Q25

How would an antialiased version of the Bresenham line look? Describe the principle.

1. true line y = mx + c
2. in each column take the 2 pixels nearest the line
3. intensity = 1 − distance (the two add to 100 %)
4. principle: aliasing can only be reduced; FSAA = render bigger then average

Example: (0,0)→(5,2): column 1: (1,0) 60 %, (1,1) 40 %; column 2: 20 %/80 % …

By hand: Fractional parts of y = m·x: with m = 2/5 → .4, .8, .2, .6. Weights are 1 − f (lower pixel) and f (upper).

Watch: Bresenham demystified (NoBS Code, 16:10) · DDA line drawing (Abdul Bari, 26:48) · Xiaolin Wu antialiased lines (NoBS Code, 10:46) · What is supersampling? (Techquickie, 4:52)

L5 · Circles, fills, triangles

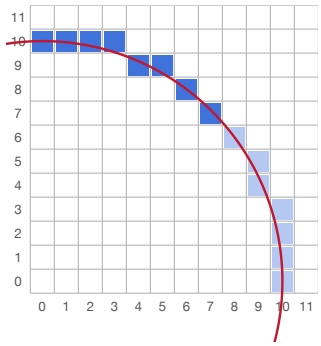
Big idea. Circle: compute one octant from (0, r) and mirror it 8 ways. Scan-line fill: per row find crossings, sort, fill between pairs. Seed (flood) fill: paint the seed, recursively visit neighbours in a fixed order. Triangles: barycentric weights α, β, γ (sum 1) say inside (all ≥ 0) and blend vertex values.

Formulas

- `circle` $p_0 = 5/4 - r$; $p < 0$: $p \mathrel{+}= 2x_+ + 1$; $p \geq 0$: $y{-}1$, $p \mathrel{+}= 2x_+ + 1 - 2y_+$
- $U = \alpha A + \beta B + \gamma C$, $\alpha + \beta + \gamma = 1$
- $f(U) = \alpha f(A) + \beta f(B) + \gamma f(C)$

Remember it

- "One octant, eight mirrors" (±x, ±y) and (±y, ±x).
- Scan-line: "Intersect, Sort, Pair, Fill". Peak or valley vertex counts twice.
- Flood order = call order. Lecture: left, right, up, down. 4-connected = rook moves.
- Barycentric weight of A = area of the small triangle opposite A ÷ whole area.



Midpoint circle P1 Q21

Describe the steps of circle drawing and mark pixels, centre (0,0), r = 10.

1. start (0, r), $p_0 = 5/4 - r$
2. $p < 0 \rightarrow (x+1, y)$, $p \mathrel{+}= 2x_+ + 1$
3. $p \geq 0 \rightarrow (x+1, y-1)$, $p \mathrel{+}= 2x_+ + 1 - 2y_+$
4. stop at $x \geq y$, mirror 8 ways

Example: (0,10) (1,10) (2,10) (3,10) (4,9) (5,9) (6,8) (7,7) → 56 pixels

Traps: using old x, y in the update (use the NEW ones) · mirroring only 4 ways

By hand: Use $p_0 = 1 - r$ (integer, same pixels as 5/4 − r): r = 10 → p = −9, −6, −1, 6, −3, 8, 5. Updates: $+2x_+{+}1$ or $+2x_+{+}1{-}2y_+$. Check: $x^2 + y^2 \approx r^2$.

Scan-line fill P1 Q22, Gracki K1

Describe the steps of the scan-line algorithm while filling the shape.

1. for each row y: crossings with the contour
2. sort the x values
3. fill between pairs (1–2, 3–4)
4. vertex rule: once if edges go to opposite sides, twice at a peak/valley; horizontal edges are not crossings

Example: "Hill" shape: rows 5–10 get 13, 12, 11, 9, 7, 5 interior pixels.

Traps: odd number of crossings (vertex counted wrongly)

By hand: Crossing $x = x_1 + (y - y_1)\cdot(\Delta x/\Delta y)$: Δx/Δy is a constant fraction per edge, so add it row by row.

Seed (starting-point) fill: order of pixels P1 Q23

Describe the starting-point algorithm and mark the order in which pixels are drawn.

1. choose and state the seed
2. state the call order (lecture: left, right, up, down)
3. recursive: paint, then call neighbours in order; a call returns when it hits the border or a painted pixel
4. number the pixels 1, 2, 3 …

Example: Seed (1,2): (1,2) (2,2) (2,3) (1,3) (3,3) (4,3) (4,4) (3,4) (4,2) (4,1)

Traps: the slide-30 picture numbers row by row, but the slide-30 code is recursive (depth-first): name your algorithm and order, then follow it · not stating the seed

By hand: No arithmetic: number the pixels while following the recursion.

Barycentric: inside test and colour P1 Q24, Q26; EXAM 1/22 #3 (texture)

Is the marked pixel inside A(0,0), B(0,4), C(4,0)? Then compute its grey from vertex values.

1. read the pixel coordinates carefully from the grid
2. solve $x = \alpha Ax + \beta Bx + \gamma Cx$, $y = \ldots$, $\alpha{+}\beta{+}\gamma = 1$
3. inside ⇔ all ≥ 0
4. value = $\alpha fA + \beta fB + \gamma fC$ (works for grey, RGB, UV)

Example: pixel (1,2): α=0.25, β=0.5, γ=0.25 → inside; grey = 0.25·255 + 0.25·128 = 95.75 ≈ 96

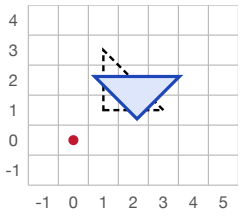
Traps: misreading the pixel (it is (1,2), not (1,1))

By hand: Right triangle at the origin, A(0,0), B(0,h), C(w,0): β = y/h, γ = x/w, α = 1 − β − γ. Otherwise use the Cramer formula with fractions. Value: a weighted sum with quarters: .25·255 = 63.75, .25·128 = 32.

Watch: Midpoint circle step by step (NoBS Code, 13:33) · Scan-line polygon fill (Priyanka Pujari, 14:04) · Flood fill (Paint bucket) (Inside code, 8:53) · Point in triangle (Sebastian Lague, 3:53)

L6 · 2D transforms and frames

Big idea. Homogeneous coordinates (x, y, 1) make translation a matrix product, so every chain of moves is one 3×3 matrix. Apply T1 then T2 → M = T2·T1 (the first is rightmost). About a point p: T(p)·X·T(−p). A point's coordinates in a rotated/moved frame: subtract the new origin, then dot with the new axes.



Formulas

- `T = [1 0 dx; 0 1 dy; 0 0 1]`
- `S = [sx 0 0; 0 sy 0; 0 0 1]`
- `R(θ) = [cos −sin 0; sin cos 0; 0 0 1] (CCW)`
- `about p: T(p)·X·T(−p)`
- `frame: u = (P−E)·û, v = (P−E)·v̂`

Remember it

- Read matrices right to left, like function composition f(g(x)).
- "Go home, spin, come back" = T(p)·R·T(−p).
- Rotation: the minus sign sits top-right for counter-clockwise.
- Rotating the frame by +θ = rotating the point by −θ.

Compose 2D transformations and apply

P2 Q1, Exam 2020 T4 (all groups), EXAM 1/23 #3

Triangle (1,1), (3,1), (1,3): translate by [1,−2], then rotate 45° CCW about the origin. Matrix first, then coordinates, then a drawing.

1. write each matrix
2. M = (last) · … · (first)
3. multiply M by [x, y, 1]^T for every vertex
4. draw before and after
5. about a point p: T(p)·X·T(−p)

Example: T[1,−2] then S(0.5,1): M = [0.5 0 0.5; 0 1 −2; 0 0 1] → (1,−1), (2,−1), (1,1)

Traps: wrong order T·S vs S·T · sin sign in R · forgetting the 1 in [x, y, 1]

By hand: Multiply matrices with integers first; keep √2/2 as a symbol for 45° and use the k·0.7071 table at the end: (1 − (−1))·√2/2 = 2·0.7071 = 1.414.

Change of coordinate frame

P2 Q2

P = (2,2) in OXY. Frame EUV is rotated 90° CCW relative to OXY. (u, v)?

1. axes of EUV in OXY: û = (cos θ, sin θ), v̂ = (−sin θ, cos θ)
2. subtract the origin E
3. u = (P−E)·û, v = (P−E)·v̂
4. = rotate the point by −θ

Example: (2, −2)

Traps: rotating the point by +θ (answer (−2, 2) is wrong)

By hand: 90° multiples need no trig: frame +90° → (u, v) = (y, −x); frame −90° → (−y, x); 180° → (−x, −y). Subtract the origin first.

Watch: Homogeneous coordinates, quickly (Miolith, 6:53) · The true power of the matrix (Computerphile, 14:46) · Matrix multiplication as composition (3Blue1Brown, 10:03) · Change of basis (3Blue1Brown, 12:50)

L6 · Clipping

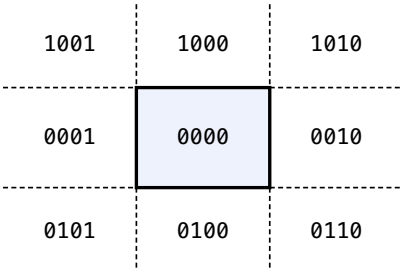
Big idea. Cohen–Sutherland gives each endpoint a 4-bit code [Top Bottom Right Left]. Both 0000 → keep it all; AND ≠ 0000 → drop it all; otherwise cut at one window edge and test again. Sutherland–Hodgman clips a polygon against one window edge at a time with 4 cases per polygon edge.

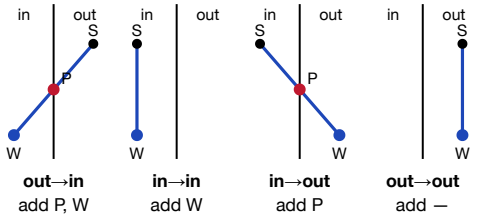
Formulas

- `code = [y>ymax, y<ymin, x>xmax, x<xmin]`
- cut at $x = c$: $y = y_1 + (y_2-y_1)(c-x_1)/(x_2-x_1)$
- cut at $y = c$: $x = x_1 + (x_2-x_1)(c-y_1)/(y_2-y_1)$
- `S–H: in→in W · in→out P · out→in P,W · out→out —`

Remember it

- "TBRL" = Top Bottom Right Left, read left to right. Top-left corner region = 1001.
- "OR = 0 accept, AND ≠ 0 reject."
- S–H: "Entering? Two points. Leaving? One point. Inside? End point. Outside? Nothing."





Cohen–Sutherland line clipping P2 Q3, Exam 2020 T3 (4 groups)

Three segments and window (4,0)–(10,5): describe the steps for each and draw.

1. draw the window and the 9 regions with codes
2. code both endpoints [T B R L]
3. both 0000 → accept; AND ≠ 0000 → reject
4. else cut the outside endpoint at one edge, recompute, repeat
5. write every intersection with its formula

Example: (5,1)–(8,3) accept · (2,7)–(6,7) reject (AND 1000) · (8,10)–(15,3): cut top → (13,5) code 0010 → AND 0010 → reject

Traps: AND = 0000 does NOT mean accept · bit order

By hand: Codes by comparison, no arithmetic. Cut points: $(c - y_1)/\Delta y$ as a fraction, then $x = x_1 + \Delta x \cdot$fraction (e.g. $(5 - 10)/(-7) = 5/7 \rightarrow x = 8 + 7 \cdot 5/7 = 13$).

Sutherland–Hodgman polygon clipping P2 Q4

Triangle (7,1), (1,4), (5,10), window (4,0)–(10,5): steps of S–H.

1. clip against one window edge at a time (left, top, right, bottom; state your order)
2. walk the polygon edges S → W; 4 cases
3. in→in: W; in→out: P; out→in: P and W; out→out: nothing
4. the output list is the input for the next edge

Example: Result: (4, 2.5), (4, 5), (6.11, 5), (7, 1)

Traps: skipping a stage that "does nothing" (still write it) · rounding 55/9 to 6

By hand: Same cut formula with fractions; 55/9 = 6.11 by long division (9·6 = 54, remainder 1 → 0.11).

Watch: Cohen–Sutherland example (Education 4u, 13:04) · Sutherland–Hodgman polygon clipping (Anmol Unpacks, 8:33)

L6 · Bézier curves

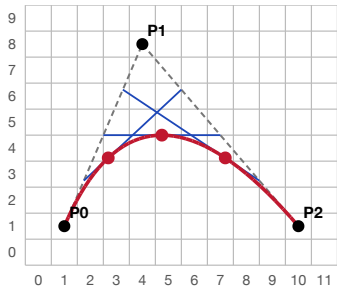
Big idea. A Bézier curve of degree n uses n + 1 control points; it starts at P0, ends at Pn and is pulled toward the inner points. De Casteljau construction: on each control segment mark the point at fraction t, join those points, repeat until one point is left: that is B(t). Smooth joining: share the end point and keep the neighbouring control points on one line.

Formulas

- `quadratic: (1−t)²P0 + 2t(1−t)P1 + t²P2`
- `cubic: (1−t)³P0 + 3t(1−t)²P1 + 3t²(1−t)P2 + t³P3`
- `C0: Q0 = P3 · G1: P2, P3, Q1 collinear · C1: Q1 = 2P3 − P2`

Remember it

- "Lerp the lines until one point is left."
- Cubic weights = Pascal row 1-3-3-1.
- "C0 touch, G1 same direction, C1 same speed."
- The curve stays inside the convex hull of its control points.



Bézier curve construction (+ smooth joining) P2 Q5, EXAM 1/22 #1

Second/third order Bézier: explain the geometric construction, show it for t = 0.25, 0.5, 0.75, draw the curve, explain smooth joining.

1. draw the control polygon
2. for each t: mark t-points on every segment, join, repeat → B(t)
3. draw the curve through P0, B(.25), B(.5), B(.75), Pn
4. smooth join: Q0 = Pn, Q1 on the line Pn−1Pn beyond Pn (C1: Q1 = 2Pn − Pn−1)

Example: Quadratic (1,1),(4,8),(10,1): (2.69,3.63), (4.75,4.5), (7.19,3.63). Cubic (1,1),(3,−7),(8,−7),(10,1): (2.97,−3.5), (5.5,−5), (8.03,−3.5)

Traps: the curve does not pass through the inner control points

By hand: t = ½ is just midpoints (averages). t = ¼, ¾: quarter points (3A + B)/4 and (A + 3B)/4. No multiplication of decimals needed.

Watch: The Beauty of Bézier Curves (Freya Holmér, 24:25) · De Casteljau's algorithm (Khan Academy / Pixar, 1:32) · Levels of continuity (Nathan Carter, 1:16)

Theory one-liners (true / false traps)

- A pixel has no size: it is a sample (position + value).
- Aliasing can only be reduced, never removed.
- Raster refresh time does not depend on the picture; a vector display does.
- Vertex shader = WHERE, fragment shader = WHAT COLOUR; GPU = SIMD, many cores in parallel.
- RGB additive (light), CMY(K) subtractive (ink); K because C+M+Y gives muddy brown.
- RGB gamut = triangle inside the horseshoe; it cannot show every colour.
- Spectral colours lie on the curved edge of the CIE diagram; purples are not spectral.
- Metamers: different spectra seen as the same colour.
- Bresenham = integers only; DDA = floating point.
- Barycentric weights interpolate colour, depth, UV (Gouraud).
- Homogeneous coordinates make translation a matrix product; $T \cdot R \neq R \cdot T$.
- Cohen–Sutherland: AND = 0000 does not mean visible.
- Sutherland–Hodgman: any polygon against a convex window.
- Bézier: global control, affine invariant, inside the convex hull.
- Implicit $f(x,y) = 0$ tests points; parametric $(x(t), y(t))$ generates them.

Last check before the test

1. Can you do one task of every type from the *Tasks and answer key* PDF without looking?
2. Do you know the conventions table on page 1 by heart?
3. Bag: printed notes, squared paper, ruler, pencils, eraser, pen, student ID. Phone off.